



T.C.

MARMARA UNIVERSITY

FACULTY of ENGINEERING

COMPUTER ENGINEERING DEPARTMENT

CSE4197 Engineering Project I Proposal

Title of the Project

A Simulation-Based Comparative Analysis of Mainframe-Class Vertical Architectures and Distributed Server Clusters Under Service-Level Constraints

Group Members

150121050 Emirhan Türkol

150121072 Kaan Yavaş

150121067 Ahmet Can Akay

Supervised by

Doç. Dr. Murat Can Ganiz

1. Aim of the Project

Computing services are deployed under explicitly defined workload characteristics and service level requirements. These requirements include throughput targets, latency constraints, availability objectives, consistency guarantees, data access patterns, and traffic variability [2], [3]. Depending on the combination of these parameters, a service may be satisfied by a single-node system, a vertically scaled architecture, or a horizontally scaled distributed cluster. Architectural suitability therefore depends on the relationship between service requirements and architectural properties, rather than on system scale alone [1].

This project focuses on the direct comparison of two architectural approaches: mainframe-class vertical architectures and distributed server clusters. The comparison is performed under identical workload characteristics and service level constraints, ensuring that both architectures are evaluated as alternative solutions to the same service definition [1], [2].

The aim of the project is to develop a simulation-based framework that accepts a fixed and explicitly defined set of service parameters, including target throughput, latency constraints specified as p95 and p99 percentiles, availability targets, required consistency level, read write ratio, data size, working set characteristics, and traffic burstiness expressed as peak to average load ratio [3], [4], [5], [6]. For each architectural approach, the framework determines the minimum cost system configuration capable of satisfying all specified constraints [1].

The framework does not produce static recommendations. Instead, it explores how the feasibility and efficiency of each architectural approach change across the defined service requirement space [1]. The objective is to identify the conditions under which each architecture satisfies the given constraints, and to characterize the resulting trade offs in performance predictability, reliability, resource utilization, and cost [1], [3]. The project aims to provide a systematic and reproducible basis for architectural evaluation that is derived directly from workload definitions and service level objectives [4], [5].

2. Methodology

This study adopts a simulation-based experimental methodology to evaluate and compare mainframe-class vertical architectures and distributed server clusters under identical workload characteristics and service level constraints [1], [2]. The methodology is structured around constructing abstract but representative system models, defining explicit input parameters, and analyzing architectural behavior through controlled simulation experiments.

Two system models are defined. The first model represents a mainframe-class vertical architecture, characterized by centralized resource management, shared memory, high input output capacity, low internal communication latency, and low failure probability [1]. System capacity in this model is represented using either abstract resource scaling parameters or a discrete set of capacity classes, allowing the simulation to select the minimum capacity configuration that satisfies the specified service constraints [1].

The second model represents a distributed server cluster composed of multiple independent nodes communicating over a network. This model includes explicit representations of data partitioning, replication, and coordination required to satisfy strong consistency and availability requirements [2], [3]. The number of nodes is treated as a configurable variable, and for each workload and service definition, the simulation determines the minimum cluster size capable of meeting all specified constraints.

A discrete event simulation approach is used to model system behavior over time. The simulator models request arrivals, CPU service, input output operations, network communication delays, coordination events, and failure and recovery processes [1], [6]. Workloads are defined using a fixed and explicit set of parameters, including target throughput, latency constraints specified as p95 and p99 percentiles, availability targets, required consistency level, read write ratio, data size, working set characteristics, and traffic burstiness expressed as peak to average load ratio [3], [4], [5], [6].

For each workload definition, the simulation evaluates candidate configurations for both architectural models and identifies those that satisfy all service

level constraints. Among the feasible configurations, the minimum cost solution is selected for each architecture [1]. Cost is modeled using an abstract total cost of ownership representation that includes system capacity costs and operational costs derived from resource utilization [1].

The outputs of the simulation include latency distributions, throughput measurements, availability metrics, resource utilization statistics, energy consumption estimates, and cost values [3], [4], [5]. These outputs are analyzed to identify architectural feasibility regions and transition points where the preferred architectural choice changes [1]. Additional controlled experiments are conducted by varying individual parameters while holding others constant, enabling isolation of the mechanisms responsible for observed behavior, such as resource saturation, coordination overhead, and tail latency effects [2], [3].

The methodology emphasizes reproducibility and transparency. All model parameters, workload definitions, and evaluation criteria are explicitly specified, and simulation experiments are conducted in a systematic manner to ensure that observed results can be interpreted and analyzed consistently [4], [5].

3. Software/Hardware Requirements

The project will be implemented using the Python programming language. Python is selected due to its suitability for developing discrete event simulations and for performing numerical analysis. Standard scientific computing libraries will be used for data processing, statistical analysis, and result visualization.

A discrete event simulation framework will be implemented either using a lightweight simulation library or a custom event scheduling mechanism. Configuration files will be used to define workload parameters, service level constraints, architectural model settings, and cost parameters. This approach ensures reproducibility and enables systematic exploration of the service requirement space.
Visualization tools will be used to generate graphs and tables representing latency distributions, resource utilization, energy consumption, cost comparisons, and architectural transition points. All simulation experiments and analyses can be executed on a standard personal computer. No specialized hardware or external computing infrastructure is required.

4. Draft Time Plan

First Semester

- Literature review on vertical system architectures, distributed server clusters, and simulation-based system evaluation.
- Formal definition of workload parameters and service level constraints.
- Design of architectural models and simulation structure.
- Implementation of the core discrete event simulation framework.

Second Semester

- Refinement of architectural models and cost representations.
- Execution of simulation experiments across defined parameter ranges.
- Analysis of simulation outputs and identification of architectural transition points.
- Preparation of figures, tables, and supporting analyse.

5. References

1. J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2019.

2. J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*, Morgan Kaufmann, 1993.

3. J. Dean and L. A. Barroso, “The Tail at Scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

4. Transaction Processing Council, *TPC-C Benchmark Specification*, Revision 5.11, 2010.

5. Transaction Processing Council, *TPC-E Benchmark Specification*, Revision 1.14, 2010.

6. M. Arlitt and C. Williamson, “Internet Web Servers: Workload Characterization and Performance Implications,” *IEEE/ACM Transactions on Networking*, vol. 5, no. 5, pp. 631–645, 1997.